

8

Making Sure Customers Find You with Search Engine Optimization

When we optimized the performance of our blog in the previous chapter, you may have noticed that the Lighthouse report also includes a **Search Engine Optimization (SEO)** score, which our app scored relatively low on. This score tells us how optimized our app is for being indexed properly and found by search engines such as Google or Bing. After successfully developing a working blog app, of course, we want our blog to be found by users. In this chapter, we are going to learn the basics of SEO and how to optimize the SEO score for our React application. Then, we are going to learn how to create meta tags for easier integration on various social media sites.

In this chapter, we are going to cover the following main topics:

- Optimizing an application for search engines
- Improving social media embeds

Technical requirements

Before we start, please install all requirements from [Chapter 1, Preparing For Full-Stack Development](#), and [Chapter 2, Getting to Know Node.js and MongoDB](#).

The versions listed in those chapters are the ones used in the book. While installing a newer version should not be an issue, please note that certain steps might work differently on a newer version. If you are having an issue with the code and steps provided in this book, please try using the versions mentioned in *Chapters 1* and *2*.

You can find the code for this chapter on GitHub:

<https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch8>.

The CiA video for this chapter can be found at:

<https://youtu.be/1xN3l0MMTbY>

If you cloned the full repository for the book, Husky may not find the `.git` directory when running `npm install`. In that case, just run `git init` in the root of the corresponding chapter folder.

Optimizing an application for search engines

Before we get started optimizing our app for search engines, let's briefly learn how search engines work. Search engines work by storing information about websites in an index. The **index** contains the location, content, and meta information of websites. Adding or updating pages in the index is called indexing and done by a crawler. A **crawler** is an automated software that fetches websites and indexes them. It is called a crawler because it follows further links on the website to find more websites. More advanced crawlers, such as the **Googlebot**, can also detect whether JavaScript is required to render the contents of a website and even render it.

The following graphic visualizes how a search engine crawler works:

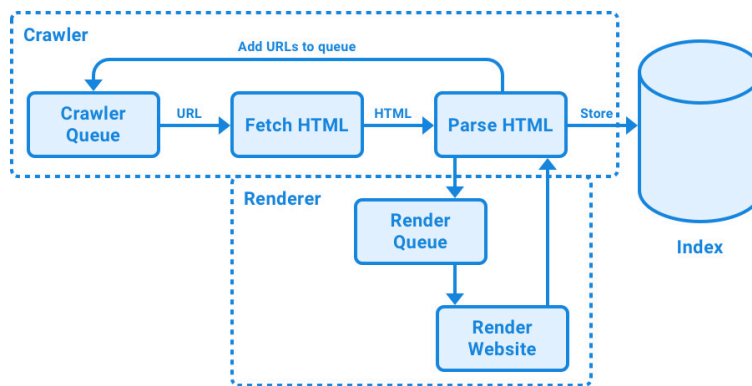


Figure 8.1 – Visualization of how a search engine crawler works

As we can see, a search crawler has a queue containing URLs that it needs to crawl and index. It then visits the URLs one by one, fetches the HTML and, if it is an advanced crawler, detects whether it needs to execute JavaScript to render the content. In that case, the URL is added to a render queue and the rendered HTML is passed back into the crawler later. Then, the crawler extracts all the links to other pages and adds them to the queue. Finally, the parsed content is added to the index.

To see whether a website is already indexed by a search engine, most search engines provide a **site:** operator, which can be used to check whether a URL is already indexed by it. For example, **site:wikipedia.org** shows various URLs on Wikipedia that are already indexed. If your website is not indexed yet, you can submit it to tools such as the **Google Search Console**. The Google Search Console also has a detailed overview of the indexing status and any problems with indexing. However, it is not necessary to submit our site for it to be found, because most search engines automatically crawl the web and will eventually find our website.

If your website still does not get indexed, this might be because it is improperly configured. First, you need to create a **robots.txt** file to specify whether search engines are allowed to crawl parts of your website, and which parts they are allowed to crawl.

NOTE

The **robots.txt** should not be used to hide web pages from Google search results. Instead, it is used to reduce traffic from crawlers on unimportant or similar pages. If you want to completely hide web pages from Google search results, either password-protect them, or use the **noindex** meta tag.

Next, you need to make sure the contents of your website are visible to the crawler. Server-side rendering can help here by allowing crawlers to view the contents of your website without running JavaScript. Additionally, adding meta information using special HTML tags helps crawlers to get additional information about your website. For small websites, pages need to be linked properly or add a manual sitemap. For larger websites, such as a blog with many posts, a sitemap should always be defined. Finally, having good performance, fast load times, and a good user experience makes your website rank higher on search engines.

We have already added server-side rendering to speed up crawling by serving content immediately without relying on JavaScript to render it. Now, let's further optimize our app for search engines. We start by creating a **robots.txt** file.

Creating a robots.txt file

First, let's ensure that crawlers are explicitly allowed to access our app and index all pages on it. To do so, we need to create a **robots.txt** file, which crawlers will read to find out which pages they are allowed to access (if any). Follow these steps to create a **robots.txt** file that allows access for all crawlers to all pages:

1. Copy the **ch7** folder to a new **ch8** folder, as follows:

```
$ cp -R ch7 ch8
```

2. Open the **ch8** folder in VS Code.
3. Create a new **public/robots.txt** file in the root of our project.
4. Open the newly created file and enter the following contents to allow all crawlers to index all pages:

```
User-agent: *  
Allow: /
```

The **robots.txt** works by defining blocks, each block being defined by matching a user agent. The user agent can match various crawlers, such as **Googlebot** for Google, or you can use ***** to match all crawlers. After the user agent, one or multiple **Allow** and/or **Disallow** statements can be made, that decide which paths a crawler is allowed or not allowed to access. In our case, we are allowing access to all paths. Additionally, a **Sitemap** can be specified, but we'll see more on that later in the *Creating a sitemap* subsection.

5. Open a Terminal pane and start the frontend by running the following command:

```
$ npm run dev
```

6. Open another Terminal pane and start the backend by running the following commands:

```
$ cd backend  
$ npm run dev
```

7. Go to **`http://localhost:5173/robots.txt`** in your browser to see the **`robots.txt`** file being served properly.

Now that we have successfully allowed crawlers to access our app, we should improve our URL structure. Let's do that by creating separate pages for each post.

Creating separate pages for posts

At the moment, it is not possible to view only a single post in our blog app, we can only view the list of all posts. That is not good for SEO, as it means a search engine will always link to the index page, which might already contain different articles than what the user was searching for. Let's refactor our app a bit to only show post titles and authors on the main page, and then link to separate pages for each blog post:

1. Edit **`src/components/Post.jsx`** to allow displaying a single full post while displaying a smaller version of the post in the list, with a link to the full version. First, we import the **`Link`** component from **`react-router-dom`**:

```
import { Link } from 'react-router-dom'
```

2. Then, we add an **`_id`** prop and a **`fullPost`** prop to the **`Post`** component. The **`fullPost`** prop will be set to **`false`** by default (when displayed in the post list) and set to **`true`** when using it in the single-post page:

```
export function Post({  
  title,  
  contents,  
  author,  
  _id,  
  fullPost = false,  
}) {
```

3. We make some adjustments to the component to show a link to the single-post page if we are not on a single-post page yet:

```
  {fullPost ? (  
    <h3>{title}</h3>  
  ) : (  
    <Link to={`/${posts}/${_id}`}>  
      <h3>{title}</h3>  
    </Link>  
  )}
```

4. Additionally, we only show the contents of the blog post on a single-post page, and adjust the spacing of the author info accordingly:

```
{fullPost && <div>{contents}</div>}
{author && (
  <em>
    {fullPost && <br />}
    Written by <User id={author} />
  </em>
)}
```

5. Adjust the prop types to add the newly defined props:

```
Post.propTypes = {
  title: PropTypes.string.isRequired,
  contents: PropTypes.string,
  author: PropTypes.string,
  _id: PropTypes.string.isRequired,
  fullPost: PropTypes.bool,
}
```

6. Edit **src/api/posts.js** and add a new function to get a single post by **id**:

```
export const getPostById = async (postId) => {
  const res = await fetch(`${import.meta.env.VITE_BACKEND_URL}/posts/${postId}`)
  return await res.json()
}
```

7. Create a new **src/pages/ViewPost.jsx** file, and start by importing all the components and functions that we are going to need:

```
import { Link } from 'react-router-dom'
import PropTypes from 'prop-types'
import { useQuery } from '@tanstack/react-query'
import { Header } from '../components/Header.jsx'
import { Post } from '../components/Post.jsx'
import { getPostById } from '../api/posts.js'
```

8. Then, define a component that accepts a **postId** as props:

```
export function ViewPost({ postId }) {
```

9. In the component, we use a query hook to fetch a single post by **id**:

```
const postQuery = useQuery({
  queryKey: ['post', postId],
  queryFn: () => getPostById(postId),
})
const post = postQuery.data
```

10. Next, render the header and a link back to the main page:

```
return (
  <div style={{ padding: 8 }}>
    <Header />
    <br />
    <hr />
```

```
<Link to='/'>Back to main page</Link>
<br />
<hr />
```

11. Then, if we managed to fetch a post with the given ID, render a post with the **fullPost** prop set. Otherwise, we show a **not found** message:

```
    {post ? <Post {...post} fullPost /> : `Post with id ${postId} not found.`}
  </div>
)
}
```

12. Lastly, define the prop types for the **ViewPost** component:

```
ViewPost.propTypes = {
  postId: PropTypes.string.isRequired,
}
```

13. Edit **src/routes.jsx** and import the **ViewPost** component and the **getPostById** function (for server-side rendering):

```
import { ViewPost } from '../pages/ViewPost.jsx'
import { getPosts, getPostById } from '../api/posts.js'
```

14. Define a new **/posts/:postId** route for viewing a single post. In the loader, we fetch the single blog post and an author, if it has one. We then return the dehydrated state and the post ID:

```
{
  path: '/posts/:postId',
  loader: async ({ params }) => {
    const postId = params.postId
    const queryClient = new QueryClient()
    const post = await getPostById(postId)
    await queryClient.prefetchQuery({
      queryKey: ['post', postId],
      queryFn: () => post,
    })
    if (post?.author) {
      await queryClient.prefetchQuery({
        queryKey: ['users', post.author],
        queryFn: () =>
          getUserInfo(post.author),
      })
    }
  }
  return { dehydratedState: dehydrate(queryClient), postId }
},
```

15. Define a **Component** method for the route, where we get **dehydrated-State** and **postId** and pass them on to the **ViewPost** component, as follows:

```
Component() {
  const { dehydratedState, postId } = useLoaderData()
  return (
    <HydrationBoundary state={dehydratedState}>
      <ViewPost postId={postId} />
    </HydrationBoundary>
  )
}
```

```
    },
  },
}
```

- Go to **http://localhost:5173/** in your browser and you will see that all blog posts in the list now have a link in their title. Click on the link to see the full blog post, as shown in the following screenshot:

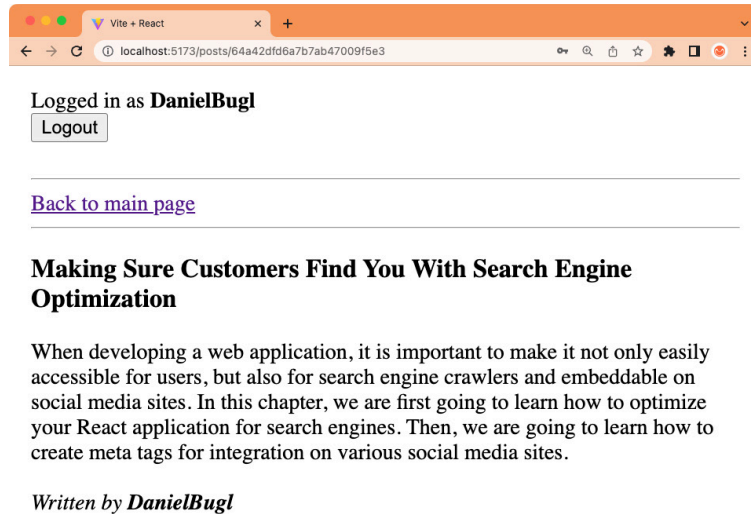


Figure 8.2 – Viewing a single blog post on a separate page

Now our blog app is already much more organized, as we do not see the full contents of all blog posts on the main page. We only see the title and author now and can then decide whether the article is interesting to us or not. Furthermore, a search engine can provide separate entries for each blog post, making it easier to find posts on our app. There is still room for improvement with the URL structure though, as it currently only contains the post ID. Let's introduce more meaningful URLs in the next step.

Creating meaningful URLs (slugs)

Websites often put keywords in the URLs to make it easier for users to see what they will be opening just by looking at the URL. Keywords in URLs are also a ranking factor for search engines, albeit a not-so-strong one. The strongest one is always good content. Nevertheless, having a good URL structure improves the user experience. For example, if the link is **/posts/64a42dfd6a7b7ab47009f5e3/making-sure-customers-find-you-with-search-engine-optimization** instead of just **/posts/64a42dfd6a7b7ab47009f5e3**, it is already clear from the URL alone what content they will find on the page. Such keywords in the URL are called a URL slug, named after “slugs” in journalism, which refers to using short descriptions of articles as internal names. Let's get started introducing slugs on our post pages:

- Edit **src/routes.jsx** and adjust the path to allow for optionally including a slug:

```
path: '/posts/:postId/:slug?',
```

NOTE

We are not doing any checks on whether the slug is correct or not. In fact, this is not really necessary, and many pages do not do this. As long as we have a correct ID, we can render the blog post. We only need to make sure that the links to the page all include the correct slug. However, we could additionally add a `<link>` element with the `rel="canonical"` attribute to a page, specifying the canonical page with the correct slug. This would tell crawlers not to index duplicate pages when incorrect slugs are used.

2. In the root of our project, install the `slug` npm package, which contains a function to properly slugify a title:

```
$ npm install slug@8.2.3
```

This package already handles unicode and returns URL-safe strings. So, we do not need to worry about sanitizing the `title` string ourselves.

3. Edit `src/components/Post.jsx` and import the `slug` function:

```
import slug from 'slug'
```

4. Then, adjust the link to the blog post by adding the slug, as follows:

```
<Link to={` /posts/${_id}/${slug(title)}`}>
```

5. Now, when we open a link from the list, the URL will look as follows:

```
http://localhost:5173/posts/64a42dfd6a7b7ab47009f5e3/making-sure-customers-find-you-with-search
```

Now we have human readable URLs for our blog posts! However, you might have noticed that the title is still **Vite + React** on all pages of our app. Let's change that now by introducing dynamic titles and including the blog post title in the page title.

Adding dynamic titles

The title of a page is even more important for SEO than keywords in the URL, as that is the title that will be shown in the search results in most cases. So, we should choose our title wisely, and if we have dynamic content (like in our blog), we should also dynamically adjust the title to fit the content. We can use the React Helmet library to facilitate changes in the `<head>` section of the HTML document. This library allows us to render a special `Helmet` component. The children of this component will replace existing tags in the `<head>` section. Follow these steps to use React Helmet to dynamically set the title:

1. First of all, let's change the general title of our app, as it is still **Vite + React**. Edit `index.html` in the root of our project and change the title. We are going to call our blog app **Full-Stack React Blog**:

```
<title>Full-Stack React Blog</title>
```

2. In the root of our project, install the `react-helmet-async` dependency to be able to dynamically change the title:


```
$ npm install react-helmet-async@1.3.0
```

NOTE

React Helmet Async is a fork of the original React Helmet that adds support for newer React versions.

3. Edit `src/pages/ViewPost.jsx` and import the `Helmet` component from `react-helmet-async`:

```
import { Helmet } from 'react-helmet-async'
```

4. Render the `Helmet` component and define the `<title>` tag inside it, as follows:

```
return (  
  <div style={{ padding: 8 }}>  
    {post && (  
      <Helmet>  
        <title>{post.title} | Full-Stack React Blog</title>  
      </Helmet>  
    )}  
  )}
```

5. Edit `src/pages/Blog.jsx` and import `Helmet`:

```
import { Helmet } from 'react-helmet-async'
```

6. Then, reset the title to `Full-Stack React Blog` in the `Blog` component:

```
return (  
  <div style={{ padding: 8 }}>  
    <Helmet>  
      <title>Full-Stack React Blog</title>  
    </Helmet>  
  )}
```

7. Edit `src/App.jsx` and import the `HelmetProvider`:

```
import { HelmetProvider } from 'react-helmet-async'
```

8. Then, adjust the `App` component to render `HelmetProvider`:

```
export function App({ children }) {  
  return (  
    <HelmetProvider>  
      <QueryClientProvider client={queryClient}>  
        <AuthContextProvider>  
          {children}  
        </AuthContextProvider>  
      </QueryClientProvider>  
    </HelmetProvider>  
  )  
}
```

9. Click on a single post in the app and you will see that the title now updates to include the post title!

Now that we have successfully set a dynamic title, let's pay some attention to other important information in the `<head>` section, the **meta tags**.

Adding other meta tags

Meta tags, as the name tells us, contain meta information about a page. Besides the title, we can set meta information such as a short description, or information on how the browser should render a website. In this section, we will cover the most important SEO-relevant meta tags, starting with the description meta tag.

Description meta tag

The description meta tag contains a short description of the contents of the page. Similarly to the title tag, we can also dynamically set this tag, as follows:

1. Edit `src/pages/Blog.jsx` and add the following generic description

`<meta>` tag:

```
<Helmet>
  <title>Full-Stack React Blog</title>
  <meta
    name='description'
    content='A blog full of articles about full-stack React development.'
  />
</Helmet>
```

Now, let's add a dynamic meta description tag for each blog post. The meta description should have between 50 and 160 characters, and since we do not have a short summary of our blog posts, let's just use the full contents and truncate them after 160 characters. Of course, it would be even better to let authors add a short summary when creating posts, but for simplicity, we just truncate the description here.

2. Edit the `src/pages/ViewPost.jsx` file and define a simple function to truncate a string:

```
function truncate(str, max = 160) {
  if (!str) return str
  if (str.length > max) {
    return str.slice(0, max - 3) + '...'
  } else {
    return str
  }
}
```

We limit the string to 160 characters, and if it's above 160, we truncate it to 157 characters and add three dots at the end.

3. Add the truncated content as a meta description tag to the `Helmet` component, as follows:

```
{post && (
  <Helmet>
    <title>{post.title} | Full-Stack React Blog</title>
    <meta name='description' content={truncate(post.contents)} />
  </Helmet>
)}
```

After adding the description meta tag, let's learn about other meta tags that could be used.

Robots meta tag

The **robots** meta tag tells crawlers whether and how they should crawl web pages. It can be used in addition to **robots.txt**, but we should only use it if we want to dynamically restrict the way a certain page is crawled. It looks as follows:

```
<meta name="robots" content="index, follow">
```

The **index** keyword tells crawlers to index the page, the **follow** keyword tells crawlers to crawl further links on the page. The **index** and **follow** keywords can be toggled off by using **noindex** and **nofollow**, respectively.

Viewport meta tag

Another important meta tag to add is the viewport tag, which tells the browser (and crawlers) that your website is mobile friendly. See the following example of how the meta tag affects how pages are rendered on mobile:

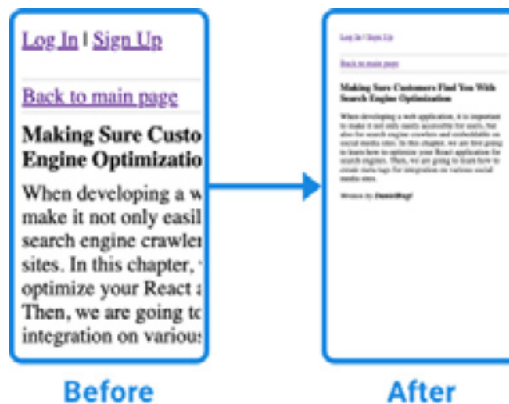


Figure 8.3 – A blog post rendering before and after adding the viewport meta tag

Vite already added this meta tag automatically for us in the **index.html** template it provided. You can see it by looking at the **index.html** file:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

After learning about the viewport tag, we continue by learning about the charset meta tag.

Charset meta tag

The charset meta tag tells the browser and crawlers about the character encoding of the web page. Usually, you want to set this to UTF-8 to ensure all Unicode characters are rendered properly. Again, Vite already added this meta tag automatically for us:

```
<meta charset="UTF-8" />
```

Now that we have learned about the relevant meta tags, let's move on to creating a sitemap, which helps crawlers find all the pages on our app more easily.

Other relevant meta information

There is additional meta information that can be relevant for a website, such as setting the language in the `<html>` tag, as follows:

```
<html lang="en">
```

Setting a favicon also improves the search snippet, which is what users see when deciding whether they should click on a link.

Creating a sitemap

A sitemap contains a list of URLs that are part of an app, so that crawlers can easily detect new content and crawl the app more efficiently. It also makes sure that all content is found, which is especially important for content-based apps with a large number of pages/posts. Usually, sitemaps are provided in XML format. They are not mandatory for SEO, but will make it easier and faster for crawlers to pick up content on your app. As we have dynamic content on our blog app, we should also create a dynamic sitemap. Follow these steps to create a dynamic sitemap for our blog app:

1. First, we are going to need a base URL for our (deployed) frontend to prefix all paths on our sitemap with. For now, we are simply going to set this to our localhost URL, but in production, this environment variable should be changed to the proper base URL of the app. Edit `.env` in the root of our project and add a `FRONTEND_URL` environment variable:

```
FRONTEND_URL="http://localhost:5173"
```

2. Create a new `generateSitemap.js` file in the root of our project, start by importing the `slug` function and `dotenv`:

```
import slug from 'slug'
import dotenv from 'dotenv'
dotenv.config()
```

3. Then, save the previously created environment variable in a `baseUrl` variable:

```
const baseUrl = process.env.FRONTEND_URL
```

4. Now, define an `async` function to generate a sitemap. In this function, we start by fetching a list of blog posts, as we want each blog post to be part of the sitemap:

```
export async function generateSitemap() {
```

```
const postsRequest = await fetch(`${process.env.VITE_BACKEND_URL}/posts`)
const posts = await postsRequest.json()
```

5. Next, we return a string containing the XML for the sitemap. We start by defining the XML header and a `<urlset>` tag:

```
return `<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
```

6. Inside the `<urlset>` tag, we can use `<url>` tags with `<loc>` tags to link to various pages. We first list all the static pages:

```
<url>
  <loc>${baseUrl}</loc>
</url>
<url>
  <loc>${baseUrl}/signup</loc>
</url>
<url>
  <loc>${baseUrl}/login</loc>
</url>
```

7. Then, we loop over all posts that we fetched from the backend and generate a `<url>` tag for each of them, constructing the URLs from the post ID and the slug:

```
${posts
  .map(
    (post) => `
<url>
  <loc>${baseUrl}/posts/${post._id}/${slug(post.title)}</loc>
```

8. We can also optionally specify a `<lastmod>` tag, telling the crawler when the content was last modified:

```
<lastmod>${post.updatedAt ?? post.createdAt}</lastmod>
```

9. Lastly, we join all generated `<url>` tags together into a single string and close the `<urlset>` tag:

```
</url>`,
)
.join('')}
</urlset>`
}
```

Now that we have a function to dynamically generate a sitemap, we still need to include a route to it in our server.

10. Edit `server.js` and import the `generateSitemap` function there:

```
import { generateSitemap } from './generateSitemap.js'
```

11. Then, go to the first `app.use('*')` declaration inside the `createProdServer` function and check whether the URL is `/sitemap.xml`. If yes, generate the sitemap and return it as XML:

```
app.use('*', async (req, res, next) => {
  if (req.originalUrl === '/sitemap.xml') {
```

```
const sitemap = await generateSitemap()
return res
  .status(200)
  .set({ 'Content-Type': 'application/xml' })
  .end(sitemap)
}
```

NOTE

In a more sophisticated setup, we could cache the generated sitemap either on our Express server, our own web server, or a separate caching service.

12. We do the same change as in the previous step for the second `app.use('*')` declaration inside the `createDevServer` function.
13. Restart the server and go to `http://localhost:5173/sitemap.xml` to see the sitemap being dynamically generated, with links to all created posts and their last modified timestamps.
14. We can now link to the sitemap in the `robots.txt` file. As an example, we are going to set the URL to localhost. In a production app, you would adjust this URL to point to the sitemap on the URL of the deployed application. Edit `public/robots.txt` and add the following line:

```
Sitemap: http://localhost:5173/sitemap.xml
```

Now that we have successfully implemented measures to improve our app for search engines, let's take a look at our SEO score in the Lighthouse report:



Figure 8.4 – Our Lighthouse SEO score is now 100!

As we can see, our SEO score is now 100 (from 91 before). This might only seem like a slight improvement, but the Lighthouse report only takes into account basic checks, such as having a title, description, viewport tag, and a `robots.txt` file. We have done much more to optimize the user experience for visitors and search engines, such as improving the URL structure and adding dynamic titles and descriptions.

We could still further optimize our app by serving static assets via a **Content Delivery Network (CDN)** and using responsive images (serving images in different sizes to optimize performance on slower connections and avoid loading the full-size images). However, that is outside the scope of this book.

To wrap up this chapter, we are going to take a look at improving embeds on social media sites.

Improving social media embeds

We have already added the important meta tags for search engines. However, social media websites, such as Facebook and X (formerly Twitter), read additional meta tags to improve the embedding of your app on their sites and apps. Most social networks use a standard called Open Graph Meta Tags, which was originally created at Facebook. These tags can contain additional information on the type of page, a special title, the description, and an image for embedding the page on a social media website.

Open Graph meta tags

The **Open Graph (OG)** meta tags have four generic properties that every page can have:

- **og:type**: Describes the type of the page; specific types may have additional properties
- **og:title**: Describes the title of the page as it should appear on embeds
- **og:image**: An URL to an image that should be used for the embed
- **og:url**: An URL to a link that should be used for the embed

The **og:type** meta tag describes the type of content available on the page. It tells the social media sites how the embed should be formatted. Among others, the following values are possible:

- **website**: The default value, a basic embed
- **article**: This is for news and blog posts, and has additional parameters for **published_time**, **modified_time**, **author**, **section**, and **tag**
- **profile**: For user profiles, with additional parameters for **first_name**, **last_name**, **username**, and **gender**
- **book**: For books, with additional parameters for **author**, **isbn**, **release_date**, and **tag**
- **music** types: This includes **music.song**, **music.album**, **music.playlist**, and **music.radio_station**, each of them having different additional parameters
- **video** types: This includes **video.movie**, **video.episode**, **video.tv_show**, and **video.other**, each of them having different additional parameters

A full description of the OG meta tags and all possible values can be found on their official website: <https://ogp.me/>.

INFO

Most social media sites support OG meta tags for embeds. However, some websites, including X (formerly Twitter), have their own meta tags, which take priority over OG meta tags, if provided. X can still read OG meta tags though, so it is enough to only provide those.

Now, we are going to focus on the **article** type, as we are developing a blog application, so we can use this type to provide better embeds for the blog posts.

Using the OG article meta tags

As we have learned, the **article** type allows us to include meta information about the published time, modified time, and author of an article on our page. Let's do this now for our single-post page:

1. Edit **src/pages/ViewPost.jsx** and import the **getUserInfo** API function, as we will need to resolve the author name for the corresponding meta tag:

```
import { getUserInfo } from '../api/users.js'
```

2. Inside the **ViewPost** component after we fetch the post, fetch the author name. We make sure to only do this call if the **post?.author** attribute exists by using the **enabled** option of the **useQuery** hook:

```
const userInfoQuery = useQuery({
  queryKey: ['users', post?.author],
  queryFn: () => getUserInfo(post?.author),
  enabled: Boolean(post?.author),
})
const userInfo = userInfoQuery.data ?? {}
```

3. Inside the **Helmet** component, we define the **og:type** tag as **article** and define the title, published time, and modified time:

```
{post && (
  <Helmet>
    <title>{post.title} | Full-Stack React Blog</title>
    <meta name='description' content={truncate(post.contents)} />
    <meta property='og:type' content='article' />
    <meta property='og:title' content={post.title} />
    <meta property='og:article:published_time' content={post.createdAt} />
    <meta property='og:article:modified_time' content={post.updatedAt} />
  </Helmet>
)}
```

4. Then, we set the **og:article:author** to the resolved username:

```
<meta property='og:article:author' content={userInfo.username} />
```

5. Lastly, we loop through the tags (if there are none, we default to an empty array) and define a meta tag for each tag:

```
{(post.tags ?? []).map((tag) => (
  <meta key={tag} property='og:article:tag' content={tag} />
))}
</Helmet>
})
```

Arrays in OG meta tags work by redefining the same property multiple times.

Now that we have successfully added meta tags, our blog app is optimized for search engines and social media sites!

Summary

In this chapter, we first briefly learned how search engines work. Then, we created a **robots.txt** file, along with separate pages for each blog

post, to better optimize our blog for search engines. Next, we created meaningful URLs (slugs) and set dynamic titles and meta tags. Then, we created a sitemap and evaluated the SEO score of our blog after all optimizations. Finally, we learned how social media embeds work and which meta tags can be used to improve embeds for articles, such as blog posts.

In the next chapter, [***Chapter 9***](#), *Implementing End-to-End Tests Using Playwright*, we are going to learn how to write end-to-end tests for our user interface by setting up Playwright. Then, we are going to write some frontend tests for our blog application.